

CSE 317: Algorithms: Design and Analysis

Project Proposal

Algorithmic Analysis of the Discrete Logarithm Problem in Cryptography

Team Members:

1. Sharique - ID: 28369
2. Maira Aijaz - ID: 28552
3. Muhammad Ibrahim - ID: 29079
4. Suffiyan Asghar Ali - ID: 29182

Date: April 17, 2026

1 Introduction and Problem Formulation

In the realm of cybersecurity, the security of many modern cryptographic protocols, such as the Diffie-Hellman Key Exchange and Elliptic Curve Cryptography (ECC), inherently relies on the computational intractability of the **Discrete Logarithm Problem (DLP)**. The problem is formally defined as follows: given a cyclic group G , a generator $g \in G$, and an element $h \in G$, find the integer x such that:

$$g^x \equiv h \pmod{p}$$

where p is a large prime number.

Our project intends to bridge theoretical algorithm design and practical cybersecurity. We will analyze how efficiently different algorithmic paradigms can solve the DLP as the size of the prime modulus p increases. By demonstrating the mathematical limits of varying algorithms, we will empirically simulate why sufficiently large primes are absolutely necessary to prevent attacks on secure communications.

2 Algorithm Description

As a group of 4 members, we have elected to implement and empirically compare the following **four distinct algorithms**, each representing a radically different algorithm design technique:

1. **Naive Brute Force Search (Exhaustive Search):** This establishes our computational baseline. It linearly computes $g^i \pmod{p}$ for $i = 1, 2, \dots, p-1$ until a match for h is found. We expect its theoretical time complexity of $O(p)$ to exponentially scale as bit-lengths increase.

2. **Baby-step Giant-step Algorithm:** This algorithm utilizes a classic time-memory trade-off. By precomputing a hash table of "baby steps" and checking "giant steps" against it, it drastically reduces the time complexity to $O(\sqrt{p})$, but incurs a massive memory footprint of $O(\sqrt{p})$.
3. **Pollard's ρ (Rho) Algorithm for Logarithms:** This probabilistic algorithm achieves the same optimized $O(\sqrt{p})$ time complexity but requires practically $O(1)$ constant memory overhead by utilizing Floyd's cycle-finding mechanism on a pseudorandom sequence.
4. **Pohlig-Hellman Algorithm:** A pure mathematical approach utilizing the Chinese Remainder Theorem (CRT). This algorithm breaks the group down into smaller subgroups based on the prime factorization of $p - 1$. It is highly efficient if $p - 1$ consists of small prime factors (smooth integers).

3 Implementation Details and Datasets

Language: We will implement our solutions in **Python** due to its native support for arbitrarily large integers (BigInts), which is crucial for cryptographic simulations without causing memory overflow errors.

Dataset Variations: We will write an automated script to generate prime numbers (p) of varying bit-lengths. Our experiments will test the algorithms across different variations:

- **Small scale:** 8-bit to 16-bit primes to verify basic algorithmic correctness.
- **Medium scale:** 24-bit to 32-bit primes to observe the precise point where Brute Force begins to fail on consumer hardware compared to the other algorithms.
- **Large scale:** 40+ bit primes to empirically test the practical limits of the $O(\sqrt{p})$ time/memory boundaries.
- **Weak Primes:** A specialized dataset where $p - 1$ has exceptionally small prime factors to explicitly showcase the structural vulnerabilities that the Pohlig-Hellman algorithm exploits.

4 Expected Results and Discussion Plan

We expect to generate robust run-time and memory-usage comparisons, presented through graphs and analytic tables. Our final report will emphasize empirical visualizers, specifically highlighting the literal Time-Memory trade-offs (Baby-step vs. Pollard's Rho) and the impact of the dataset structure (Pohlig-Hellman on smooth vs non-smooth primes). We will conclude with insights into how modern cybersecurity protocols select parameters to thwart these mathematical attacks.